Student can select any one program as per their feasibility here number of options provided.

REFERENCES :-
HTTPS://WWW.TOPCODER.COM/THRIVE/ARTICLES/IMPLEMENTING-GO-BACK-N-PROTOCOL-IN-C

WHAT IS GO BACK-N

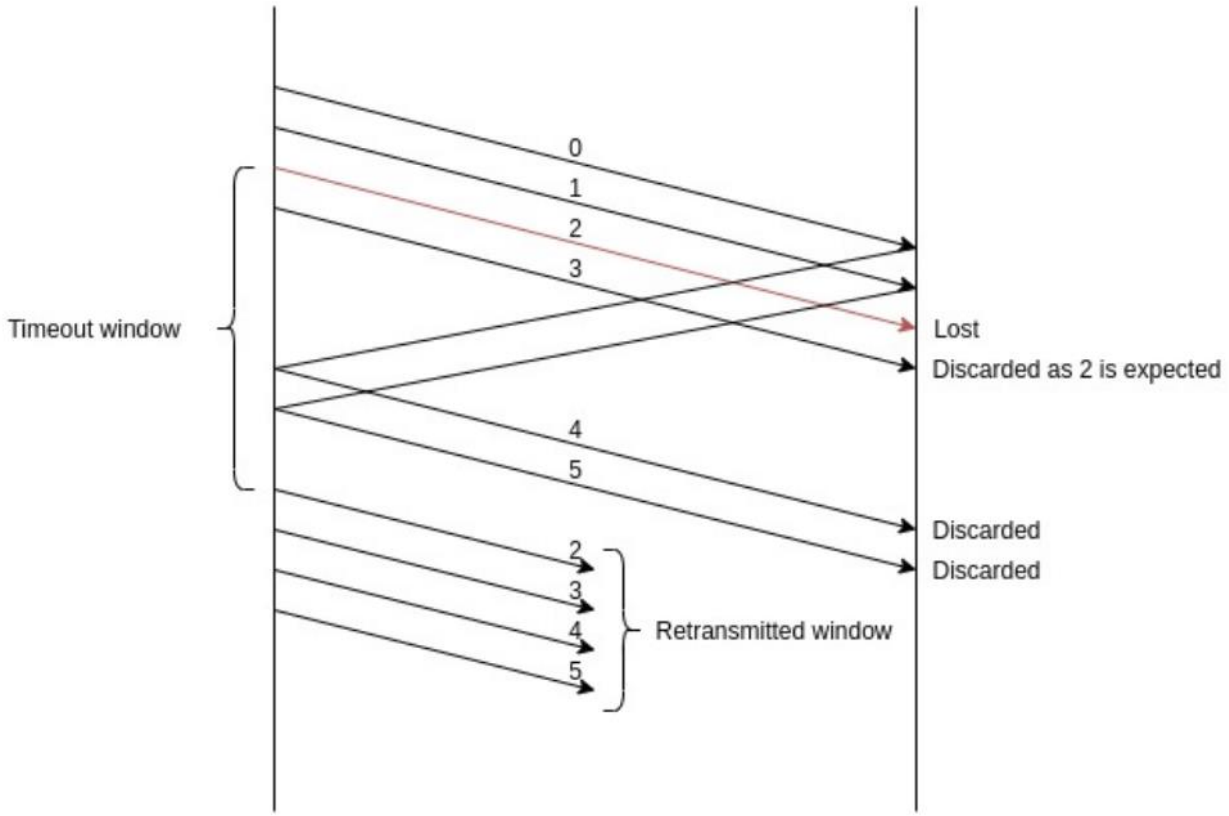
The Go Back-N ARQ or Go Back Automatic Repeat Request is a way to implement sliding window protocol. This protocol is used for flow control in networking and is a part of the data-link layer. The sender's window is of size N and the receiver's window size is always one. The window sizes for Go Back-N are:

1. SENDER WINDOW SIZE (WS)

The WS is equal to N. If we say the protocol is GB-4, then WS = 4. In order to implement pipelining, the window size should always be greater than one. If window size is one, the protocol reduces to stop-and -wait protocol.

2. RECEIVER WINDOW SIZE (WR)

WR is always one in Go Back-N.
Now we will understand how Go Back-N actually works with the help of an example. In the diagram below the sender's window has a size of four, or Go Back-4. Assume that we've plenty of sequence numbers, for the sake of explanation. Now the sender has despatched the packets zero, one, two and three. After acknowledging the packets zero and one, the receiver is now awaiting packet two and the sender window has additionally slided to similarly transmit the packets four and five.
Now suppose packet two is misplaced within the network. The receiver will discard all the packets which the sender has transmitted after packet two as it's awaiting the packet with sequence number two. On the sender side, for every packet dispatched there is a time-out timer to expire for packet range two. Now from the remaining transmitted packet five the sender will go back to the packet with sequence number two within the current window and transmit all the packets until packet number five. That's why it is referred to as Go Back-N. In the Go Back-N approach the sender has to move back N places from the closing transmitted packet within the unacknowledged window and not from the factor where the packet is misplaced.



3. **ACKNOWLEDGEMENTS**

There are 2 kinds of acknowledgments, namely:

- Cumulative ACK
- Independent ACK

GBN makes use of cumulative acknowledgement. On the receiver side, it starts an acknowledgment timer when any packet is received, which is constant, and when it expires, it's going to send a cumulative ACK for the range of packets received in the interval of the timer. If the receiver has obtained N packets, then the acknowledgement range might be N+1. A crucial factor is that the ACK timer will now not begin after the expiration of the primary timer, but after the receiver has received a packet. The timer at the sender side must be greater than the ACK timer.

CODE:

```
#include<bits/stdc++.h>

#include<ctime>

#define ll long long int
using namespace std;

void transmission(ll & i, ll & N, ll & tf, ll & tt) {
  while (i <= tf) {
    int z = 0;
    for (int k = i; k < i + N && k <= tf; k++) {
      cout << "Sending Frame " << k << "..." << endl;
      tt++;
    }
    for (int k = i; k < i + N && k <= tf; k++) {
      int f = rand() % 2;
      if (!f) {
        cout << "Acknowledgment for Frame " << k << "..." << endl;
        z++;
      } else {
        cout << "Timeout!! Frame Number : " << k << " Not Received" << endl;
        cout << "Retransmitting Window..." << endl;
        break;
      }
    }
    cout << "\n";
    i = i + z;
  }
}

int main() {
  ll tf, N, tt = 0;
  srand(time(NULL));
  cout << "Enter the Total number of frames : ";
  cin >> tf;
  cout << "Enter the Window Size : ";
  cin >> N;
  ll i = 1;
  transmission(i, N, tf, tt);
  cout << "Total number of frames which were sent and resent are : " << tt <<
    endl;
  return 0;
}
```

OUTPUT:

```
Enter the Total number of frames : 12
Enter the Window Size : 4
Sending Frame 1…
Sending Frame 2…
Sending Frame 3…
Sending Frame 4…
Timeout!! Frame Number : 1 Not Received
Retransmitting Window…

Sending Frame 1…
Sending Frame 2…
Sending Frame 3…
Sending Frame 4…
Acknowledgment for Frame 1…
Timeout!! Frame Number : 2 Not Received
Retransmitting Window…

Sending Frame 2…
Sending Frame 3…
Sending Frame 4…
Sending Frame 5…
Timeout!! Frame Number : 2 Not Received
Retransmitting Window…

Sending Frame 2…
Sending Frame 3…
Sending Frame 4…
Sending Frame 5…
Acknowledgment for Frame 2…
Acknowledgment for Frame 3…
Acknowledgment for Frame 4…
```

Timeout!! Frame Number : 5 Not Received
Retransmitting Window…

Sending Frame 5…
Sending Frame 6…
Sending Frame 7…
Sending Frame 8…
Timeout!! Frame Number : 5 Not Received
Retransmitting Window…

Sending Frame 5…
Sending Frame 6…
Sending Frame 7…
Sending Frame 8…
Acknowledgment for Frame 5…
Timeout!! Frame Number : 6 Not Received
Retransmitting Window…

Sending Frame 6…
Sending Frame 7…
Sending Frame 8…
Sending Frame 9…
Acknowledgment for Frame 6…
Timeout!! Frame Number : 7 Not Received
Retransmitting Window…

Sending Frame 7…
Sending Frame 8…
Sending Frame 9…
Sending Frame 10…
Acknowledgment for Frame 7…
Acknowledgment for Frame 8…
Acknowledgment for Frame 9…
Acknowledgment for Frame 10…

Sending Frame 11…
Sending Frame 12…
Timeout!! Frame Number : 11 Not Received
Retransmitting Window…

Sending Frame 11…
Sending Frame 12…
Timeout!! Frame Number : 11 Not Received
Retransmitting Window…

Sending Frame 11…
Sending Frame 12…
Acknowledgment for Frame 11…
Acknowledgment for Frame 12…

Total number of frames transmitted(sent + resent) are : 38

EXPLANATION:

Here we are taking input for two values, the window size and the number of frames to be transmitted, with variable names N and tf respectively. We have also taken the variable tt to count total transmitted/retransmitted frames. The values of window size and number of frames are passed to the transmitter function, which in turn sends frames from frame number i to i+N (current frame to current frame + window size).

It then waits for acknowledgement, which is simulated here with a random number generator function which generates zero, one depicting the ACK is received or lost.

If ACK is not received, it retransmits the window after it.
These steps are run again and again until all frames are transmitted.

OBSERVATION:

In our case we transmitted frames one, two, three, and four. Acknowledgement for frame one was not received six times, so frames one through four were sent a total of seven times. Then ACK for frame seven was lost so windows seven, eight, nine, and ten were retransmitted for which we didn't receive acknowledgement for frame nine. Then nine, ten, eleven, and twelve windows were re-sent and we received ACK for it, completing our transmission.

OR

You can refer this code also.

Reference : - https://onecompiler.com/c/3wwh28fk7

```
//Pgm-2: Go-Back-N ARQ & Selective Repeat ARQ Sliding Window Protocol
#include<stdio.h>
#include<stdlib.h>
#include<math.h>
#include<unistd.h>
int n,r;
struct frame
{
char ack;
int data;
}frm[10];
int sender(void);
void recvack(void);
void resend(void);
void resend1(void);
void goback(void);
```

```
void selective(void);
int main()
{
int c;
do
{
printf("\n\n1.Selective repeat ARQ\n2.Goback ARQ\n3.exit");
printf("\nEnter your choice:");
scanf("%d",&c);
switch(c)
{
case 1:selective();
break;
case 2:goback();
break;
case 3:exit(0);
break;
}
}while(c>=4);
}
void goback()
{
sender();
recvack();
resend1();
printf("\n all packets sent successfully\n");
}
void selective()
{
```

```
sender();
recvack();
resend();
printf("\nAll packets sent successfully");
}
int sender()
{
int i;
printf("\nEnter the no. of packets to be sent:");
scanf("%d",&n);
for(i=1;i<=n;i++)
{
printf("\nEnter data for packets[%d]",i);
scanf("%d",&frm[i].data);
frm[i].ack='y';
}
return 0;
}
void recvack()
{
int i;
rand();
r=rand()%n;
frm[r].ack='n';
for(i=1;i<=n;i++)
{
if(frm[i].ack=='n')
printf("\nThe packet number %d is not received\n",r);
}
```

```
}
void resend() //SELECTIVE REPEAT
{
printf("\nresending packet %d",r);
sleep(2);
frm[r].ack='y';
printf("\nThe received packet is %d",frm[r].data);
}
void resend1() //GO BACK N
{
int i;
printf("\n resending from packet %d",r);
for(i=r;i<=n;i++)
{
sleep(2);
frm[i].ack='y';
printf("\nReceived data of packet %d is %d",i,frm[i].data);
}
}
```

OR

SLIDING WINDOW PROTOCOL (GO BACK N)

AIM: To write a program to perform simulation on sliding window protocol.

ALGORITHM:

Step 1: Start the program.

Step 2: Generate a random that gives the total number of frames to be transmitted.

Step 3: Set the size of the window.

Step 4: Generate a random number less than or equal to the size of the current window and identify the number of frames to be transmitted at a given time.

Step 5: Transmit the frames and receive the acknowledgement for the frames sent.

Step 6: Find the remaining frames to be sent.

Step 7: Find the current window size.

Step 8: If an acknowledgement is not received for a particular frame retransmit the frames from that frame again.

Step 9: Repeat the steps 4 to 8 till the number of remaining frames to be send becomes zero.

Step 10: Stop the program.

CODING:

```
#include<stdio.h>
 #include<conio.h>
#include<stdlib.h>
 void main()
 {
 int temp1,temp2,temp3,temp4,i,winsize=8,noframes,moreframes;
char c;
int reciever(int);
int simulate(int);
clrscr();
 temp4=0,temp1=0,temp2=0,temp3=0;
for(i=0;i<200;i++)
```

```
 rand();
 noframes=rand()/200;
printf("n number of frames is %d",noframes);
 getch();
moreframes=noframes;
 while(moreframes>=0)
{
 temp1=simulate(winsize);
 winsize-=temp1;
 temp4+=temp1;
if(temp4 >noframes)
 temp4 = noframes;
 for(i=temp3+1;i<=temp4;i++)
 printf("nsending frame %d",i);
}
getch();
}
```